NESNE TABANLI PROGRAMLAMA

Laboratuvar Föyü

Hafta 6 — Operatör Aşırı Yükleme, İndeksleyiciler, Yapılar ve Enum

Öğrenci Yönergesi — Adım Adım Uygulama

Bilgi	Değer
Ders	Nesne Tabanlı Programlama (2. Sınıf)
Konu	Operatör overloading · indexer · struct · enum
Ortam	Visual Studio Community 2026 · .NET 10 (LTS)
Dil	C#
Süre	Yaklaşık 2–3 saat
Teslim	Pdf-word

1. Bu Haftanın Hedefi

Bu hafta kendi sınıflarımıza/yapılarımıza operatörlerle (+, *, ==) yeni anlamlar yükleyeceğiz. Bir Kesir (rasyonel sayı) yapısı tasarlayıp iki kesri doğrudan k1 + k2 gibi toplayabilecek, ayrıca indeksleyici (this[i]) ve enum sabitleriyle bu yapıyı daha kullanışlı hâle getireceğiz.

2. Kazanımlar

Bu haftanın sonunda şunları yapabilir hâle geleceksiniz:

- Aritmetik operatörleri (operator +, operator *) aşırı yüklemek,
- İlişkisel operatörleri (operator ==, operator !=) birlikte aşırı yüklemek,
- struct (yapı) tanımlayıp değer tipi olarak kullanmak,
- İndeksleyici (this[int]) ile bir sınıfın iç dizisine erişmek,
- enum sabitleriyle okunabilir seçenekler tanımlamak.

3. Temel Kavramlar

3.1. Operatör Aşırı Yükleme

Kendi tipimiz için bir operatöre yeni anlam yükleyen özel bir metottur. Kurallar: metot static olur, adında operator anahtar sözcüğü geçer ve parametrelerden en az biri o sınıf/yapı türünden olmalıdır.

```
public static DonusTipi operator op(parametreler)
{ ... return ...; }
```

3.2. struct (Yapı)

Sınıfa çok benzer biçimde tanımlanan, ancak değer tipi olan yapıdır; struct anahtar sözcüğüyle tanımlanır. Birbiriyle ilişkili az sayıda veriyi bir arada tutmak için uygundur. struct içinde de operatör aşırı yükleme ve metot yazılabilir.

3.3. İndeksleyici (Indexer)

Bir nesneye, dizi gibi köşeli parantezle (nesne[i]) erişmeyi sağlar. Özelliğe benzer; farkı, parametresinin köşeli parantez içinde yazılmasıdır ve ada değil this anahtar sözcüğüne dayanır:

```
public ElemanTipi this[int indeks]
{
    get { return dizi[indeks]; }
    set { dizi[indeks] = value; }
}
```

3.4. enum (Numaralandırma)

Belli bir küme sabiti okunabilir isimlerle tanımlar. Varsayılan değerleri 0'dan başlar ve birer artar:

```
enum Islem { Toplama, Carpma }   // Toplama=0, Carpma=1
```

4. Uygulama: Kesir (Rasyonel Sayı) Yapısı

Pay ve paydadan oluşan bir Kesir yapısı yazacak; toplama, çarpma ve eşitlik operatörlerini aşırı yükleyeceğiz.

Önce **Hafta6_Kesirler** adında bir Konsol Uygulaması (C#, .NET 10) oluşturun ve **Kesir.cs** dosyasını ekleyin.

Adım A — Kesir yapısı ve kurucu

Payda olarak 0 verilirse sıfıra bölmeyi önlemek için 1'e sabitliyoruz:

```
using System;

namespace Hafta6_Kesirler
{
    struct Kesir
    {
        public int Pay;
        public int Payda;

        public Kesir(int pay, int payda)
        {
            Pay = pay;
            Payda = (payda == 0) ? 1 : payda;   // sıfıra bölmeyi engelle
        }

        public void Yaz()
        {
            Console.WriteLine($"{Pay}/{Payda}");
        }
    }
}
```

Adım B — Toplama ve çarpma operatörleri

İki kesrin toplamı: a/b + c/d = (a·d + c·b) / (b·d). Çarpım ise: a/b · c/d = (a·c)/(b·d). Bu metotları struct'ın içine ekleyin:

```
public static Kesir operator +(Kesir a, Kesir b)
{
    int pay = a.Pay * b.Payda + b.Pay * a.Payda;
    int payda = a.Payda * b.Payda;
    return new Kesir(pay, payda);
}

public static Kesir operator *(Kesir a, Kesir b)
{
    return new Kesir(a.Pay * b.Pay, a.Payda * b.Payda);
}
```

Adım C — Eşitlik operatörleri (== ve !=)

İki kesir, çapraz çarpımları eşitse eşittir (ör. 2/4 ile 1/2). İlişkisel operatörler çiftler hâlinde yüklenir: operator == yazıldıysa operator != de yazılmalıdır:

```
public static bool operator ==(Kesir a, Kesir b)
{
    return a.Pay * b.Payda == b.Pay * a.Payda;   // çapraz çarpım
}

public static bool operator !=(Kesir a, Kesir b)
{
    return !(a == b);
}
```

⚠Derleyici uyarısı (hata değil)
== ve != yüklediğinizde derleyici, Equals ve GetHashCode metotlarını da yazmanızı öneren bir UYARI verebilir. Bu bir hata değildir; programınız derlenir ve çalışır. İleri konu olduğu için şimdilik görmezden gelebilirsiniz.

Adım D — Main'de test

```
using System;

namespace Hafta6_Kesirler
{
    class Program
    {
        static void Main(string[] args)
        {
            Kesir k1 = new Kesir(1, 2);
            Kesir k2 = new Kesir(1, 3);

            Console.Write("k1 + k2 = ");
            (k1 + k2).Yaz();
            Console.Write("k1 * k2 = ");
            (k1 * k2).Yaz();

            Console.WriteLine($"k1 == k2 ? {k1 == k2}");

            Kesir k3 = new Kesir(2, 4);
            Kesir k4 = new Kesir(1, 2);
            Console.WriteLine($"2/4 == 1/2 ? {k3 == k4}");
        }
    }
}
```

Ne görmelisiniz?
k1 + k2 = 5/6
k1 * k2 = 1/6
k1 == k2 ? False
2/4 == 1/2 ? True

5. Kendi Göreviniz (Lab Teslimi)

Uygulamayı aşağıdaki gibi genişletin. Teslim edilecek çalışma budur:

1. Bir KesirKutusu sınıfı yazın; içinde bir Kesir[] dizisi tutsun ve this[int] indeksleyicisiyle elemanlarına erişilsin (get ve set).
2. Bir enum Islem { Toplama, Carpma } tanımlayın.
3. Seçilen Islem değerine göre iki kesri toplayan veya çarpan bir metot yazın (parametre olarak enum alsın).

4. Kutuya en az 3 kesir koyun, indeksleyiciyle okuyun; hem toplama hem çarpma sonucunu ekrana yazdırın.

6. Sık Yapılan Hatalar

- **Operatör metodunu static yapmamak:** Operatör metotları mutlaka public static olmalıdır.
- **== yazıp != yazmamak:** İlişkisel operatörler zıt çiftler hâlinde yüklenir; biri eksikse derleme hatası alırsınız.
- **Paydayı 0 bırakmak:** Sıfıra bölme çalışma zamanı hatasıdır; kurucuda kontrol ettik.
- **İndeksleyiciyi normal parantezle tanımlamak:** İndeksleyici parametresi köşeli parantez içine yazılır: this[int i].
- **enum'a nesneyle erişmek:** enum sabitine Islem.Toplama biçiminde, tür adıyla erişilir.

7. Değerlendirme

Ölçüt	Puan
Proje derleniyor ve çalışıyor	15
Kesir yapısı (struct) + kurucu doğru	15
operator + ve operator * doğru çalışıyor	25
operator == ve != birlikte tanımlı	15
Görev: indeksleyici (this[int])	15
Görev: enum Islem + işlem metodu	15
TOPLAM	100

8. Ödev (Gelecek Haftaya Hazırlık)

Kesir yapısına çıkarma (operator -) ve bölme (operator /) operatörlerini de ekleyin. Bölmede a/b ÷ c/d = a/b × d/c kuralını kullanın.

Not: Kesirleri sadeleştirmek (ör. 2/4 → 1/2) için paydanın ve payın en büyük ortak bölenine (EBOB) bölünmesi gerekir; bunu düşünmeye başlayın. İleride koleksiyonlar ve dosya işlemleriyle bu yapıları saklamayı da göreceğiz.